

HumAIne Technical Reference ,Ã Active Learning & XAI
Knowledge Base Document for HumAIne Swarm Chatbot (RAG Indexing)

****Document purpose:**** This document is designed to be indexed into the HumAIne Swarm chatbot's RAG knowledge base. It enables the chatbot to answer precise technical questions from students and developers working on Active Learning and XAI tasks within the HumAIne ecosystem. All code examples use the exact variable names from the HumAIne hackathon notebook and are aligned with HumAIne architectural principles.

****Schema version:**** 1.1 (covers AL + XAI combined session)

****Related HumAIne deliverables:**** D5.1 (Platform Integration), D5.4 (Training Centre), HumAL README

****Related pilot:**** HumAIne smart ticketing pilot (UC: automated ticket routing with human-in-the-loop)

Part 1 ,Ã HumAIne Context and Design Philosophy

1.1 Why Active Learning in HumAIne

HumAIne is built on the Human-in-the-Loop (HITL) and Human-in-Command (HIC) paradigms. Active Learning (AL) is one of the core collaborative AI paradigms in HumAIne because it operationalises HITL in a concrete way: the AI system decides which examples it is most uncertain about, and a human provides the labels. This creates a feedback loop where human judgment directly shapes model improvement.

In the HumAIne ticketing pilot (HumAL), Active Learning is used to train a ticket routing classifier with minimal labeling effort. Agents query the most informative unlabeled tickets, a human operator labels them, and the classifier improves iteratively. This is documented in D5.4 Module 3: Advanced AI Techniques for Collaborative Systems.

1.2 Why XAI in HumAIne

Explainability is a first-class requirement in HumAIne, not an afterthought. The EU AI Act and HumAIne's trustworthy AI principles require that AI routing decisions be transparent and auditable. The `humaine-explainerdashboard` library provides SHAP-based global explanations (which features matter most across all predictions) and LIME-based local explanations (which words drove a specific routing decision). Together they satisfy Module 3's explainability requirements (LIME, SHAP, PDP) documented in D5.4.

1.3 The HumAIne Ticketing Dataset Context

In the hackathon and in the HumAL pilot, support tickets are routed to one of four teams:

- ****Hardware Team**** (`comp.sys.ibm.pc.hardware`)
- ****Software Team**** (`comp.os.ms-windows.misc`)
- ****Design Team**** (`comp.graphics`)
- ****Electronics Team**** (`sci.electronics`)

Text is vectorized using TF-IDF (3000 features, English stop words removed). The cold-start problem is simulated with only 12 labeled tickets (3 per team).

Part 2 ,Ã Active Learning with modAL

2.1 Library Overview

`modAL` (Modular Active Learning) is the AL library used in HumAIne hackathon tasks. The central class is `ActiveLearner`, which wraps any scikit-learn compatible classifier and adds query strategy capabilities.

```
```python
from modAL.models import ActiveLearner
from modAL.uncertainty import uncertainty_sampling, margin_sampling,
entropy_sampling
```
```

2.2 Choosing the Right Classifier

****For HumAIne text classification tasks, the recommended classifier is `LogisticRegression`.****

```
```python
from sklearn.linear_model import LogisticRegression
MY_CLASSIFIER = LogisticRegression(max_iter=1000)
```
```

****Rationale aligned with HumAIne design principles:****

- TF-IDF vectors are high-dimensional (3000+ features) and sparse. LogisticRegression is the standard choice for this feature space.
- It natively supports `predict\_proba()` which is required by all uncertainty-based query strategies.
- It is also required for the XAI pipeline: `ClassifierExplainer` with a LogisticRegression selects `shap.LinearExplainer`,Ã fast, accurate, and appropriate for linear models.
- Training time is fast (seconds), enabling the iterative AL loop to run efficiently in a notebook.

****When NOT to use other classifiers:****

- `SVC(kernel='rbf', probability=True)`: Accurate but slow ,Ã Platt scaling for probabilities adds significant overhead. Use only if accuracy is the sole concern and time is not a constraint.
- `RandomForestClassifier`: Suboptimal on sparse TF-IDF. Random forests work best on dense, tabular data with meaningful feature interactions.

2.3 Choosing the Right Query Strategy

****For HumAIne multi-class text classification with few initial labels, the recommended strategy is `entropy\_sampling`.****

```

```python
from modAL.uncertainty import entropy_sampling
MY_QUERY_STRATEGY = entropy_sampling
```

**Rationale aligned with HumAIne design principles:**
- With 4 teams and only 12 initial labels (3 per team), the model needs to cover uncertainty across ALL classes simultaneously.
- `entropy_sampling` selects the sample with the highest Shannon entropy over all class probability distributions ,Ã it is the only strategy that inherently handles multi-class uncertainty comprehensively.
- `uncertainty_sampling` considers only the maximum class probability (most uncertain overall) ,Ã less informative in multi-class settings.
- `margin_sampling` considers only the gap between the top-2 classes ,Ã misses uncertainty contributions from classes 3 and 4.

**When to use other strategies:**
- `uncertainty_sampling`: Acceptable for binary classification or when classes are well-separated except for one boundary.
- `margin_sampling`: Useful when the problem is effectively binary at the decision boundary (two dominant classes).

```

2.4 Choosing Queries Per Round

Recommended: 10 queries per round.

```

```python
N_QUERIES_PER_ROUND = 10
```

**Rationale:**
- With 12 initial labels and 10 rounds:  $12 + (10 \times 10) = 112$  total labeled tickets.
- This gives approximately 28 tickets per team ,Ã sufficient for a robust classifier.
- Too few (5): convergence is slow, 10 rounds yield only 62 total labels.
- Too many (30): the model loses selectivity ,Ã large batches query samples that are not maximally informative.
- In the HumAIne HITL framework, smaller batches also mean more frequent human feedback loops, which is conceptually preferable.

```

2.5 ActiveLearner Constructor

```

```python
learner = ActiveLearner(
 estimator=MY_CLASSIFIER, # sklearn classifier instance
 query_strategy=MY_QUERY_STRATEGY, # callable strategy function
 X_training=X_initial, # initial labeled features (numpy
array, shape: 12 x 3000)
 y_training=y_initial # initial labeled targets (numpy
array, shape: 12,)
)
```

```

```

**Important notes:**
- The `ActiveLearner` calls `estimator.fit(X_training, y_training)` automatically on initialization.
- Pass a classifier **instance** (with parentheses and parameters), not the class itself.
- Pass the strategy **function** (no parentheses), not a string.
- After initialization, the underlying sklearn model is accessible as `learner.estimator`.

```

```

**Verification:** After initialization, check:
```python
baseline_acc = accuracy_score(y_test, learner.predict(X_test))
Expected: 0.35 ,Äi 0.55 (12 labels is very few)
```

```

2.6 One Complete Query-Teach Cycle

This is the core operation of Active Learning. It must be implemented correctly for the loop to work.

```

```python
Step 1 ,Äi Query: which samples does the model want labeled?
query_idx, query_instance = learner.query(X_pool,
n_instances=N_QUERIES_PER_ROUND)
query_idx: integer array of indices into X_pool (shape:
N_QUERIES_PER_ROUND,)
query_instance: feature vectors for queried samples (same as
X_pool[query_idx])

Step 2 ,Äi Retrieve labels (in simulation, true labels are in y_pool)
new_labels = y_pool[query_idx]
new_labels: integer array of team indices (shape: N_QUERIES_PER_ROUND,)

Step 3 ,Äi Teach: update the model with newly labeled samples
learner.teach(X_pool[query_idx], new_labels)
Internally: re-fits the classifier on all labeled data seen so far

Step 4 ,Äi Remove queried samples from pool (CRITICAL ,Äi do both
arrays)
X_pool = np.delete(X_pool, query_idx, axis=0)
y_pool = np.delete(y_pool, query_idx, axis=0)
axis=0 removes rows (samples); omitting axis=0 removes columns
(features) ,Äi wrong
```

```

```

**Common errors:**
- Using `axis=1` or omitting `axis` in `np.delete` ,Äi removes feature
columns instead of sample rows.
- Updating `X_pool` but forgetting `y_pool` ,Äi causes index mismatch on
the next query.
- Passing `query_instance` to `learner.teach()` instead of
`X_pool[query_idx]` ,Äi both work, but `X_pool[query_idx]` is more
explicit.

```

- Not calling `np.delete` at all ,Ã the pool never shrinks and the same samples are queried repeatedly.

2.7 Full AL Loop (automated in Cell 4)

```
```python
for i in range(N_REMAINING):
 n_q = min(N_QUERIES_PER_ROUND, len(X_pool))
 if n_q == 0:
 break
 query_idx, _ = learner.query(X_pool, n_instances=n_q)
 learner.teach(X_pool[query_idx], y_pool[query_idx])
 X_pool = np.delete(X_pool, query_idx, axis=0)
 y_pool = np.delete(y_pool, query_idx, axis=0)
 acc = accuracy_score(y_test, learner.predict(X_test))
...

```

**\*\*Expected accuracy trajectory with LogisticRegression + entropy\_sampling + 10 queries/round:\*\***

- Baseline (12 labels): 0.35,Ã0.55
- After Round 1 (22 labels): 0.60,Ã0.70
- After Round 5 (62 labels): 0.75,Ã0.82
- After Round 10 (112 labels): 0.80,Ã0.85

---

## Part 3 ,Ã XAI with humane-explainerdashboard and LIME

### 3.1 Library Overview

`humane-explainerdashboard` is HumAIne's XAI framework, providing:

- **\*\*`ClassifierExplainer`\*\***: wraps a trained sklearn classifier, computes SHAP values, exposes Plotly-based explanation plots
- **\*\*SHAP backend auto-selection\*\***: `LinearExplainer` for linear models, `TreeExplainer` for tree models, `KernelExplainer` as fallback
- **\*\*Plot methods\*\***: `plot\_importances()`, `plot\_shap\_contributions()`, `plot\_confusion\_matrix()`, and more

`lime` provides `LimeTextExplainer` for local text explanations ,Ã explains individual predictions by perturbing word presence.

```
```python
from explainerdashboard import ClassifierExplainer
from lime.lime_text import LimeTextExplainer
...

```

3.2 ClassifierExplainer Constructor

****The key distinction: pass `learner.estimated`, not `learner`.****

```
```python
explainer = ClassifierExplainer(
 learner.estimated, # the underlying LogisticRegression ,Ã NOT the
 ActiveLearner

```

```

 X_test_df, # pandas DataFrame with named columns
(feature_names as columns)
 y_test_series, # pandas Series of true integer labels
 labels=TEAM_NAMES # ['Hardware', 'Software', 'Design',
'Electronics']
)
 ...

```

**\*\*Why `learner.estimator` and not `learner`:\*\***

- `ClassifierExplainer` inspects the model type using `isinstance()` checks to select the SHAP backend.
- A `modAL.ActiveLearner` is not a recognized sklearn estimator type ,Ã it would trigger `KernelExplainer` (very slow, minutes per computation).
- `learner.estimator` returns the actual `LogisticRegression` object, which triggers `shap.LinearExplainer` ,Ã fast and mathematically exact for linear models.

**\*\*Why `X\_test\_df` (DataFrame) and not `X\_test` (numpy array):\*\***

- `ClassifierExplainer` requires named feature columns for SHAP attribution plots.
- `X\_test\_df` is a pandas DataFrame with `feature\_names` as column headers (3000 TF-IDF vocabulary terms).
- It is already prepared in Cell 2: `X\_test\_df = pd.DataFrame(X\_test\_np, columns=feature\_names)`

**\*\*Expected initialization output:\*\***

```

...

```

```

Note: shap values for shap='linear' get calculated against
X_background...
Generating self.shap_explainer = shap.LinearExplainer(model, X)...
...

```

This is normal. The LinearExplainer is the correct backend for LogisticRegression.

### ### 3.3 SHAP Global Feature Importance Plot

```

...python
fig = explainer.plot_importances()
fig.show()
...

```

**\*\*What this shows:\*\*** Mean absolute SHAP value per TF-IDF feature across all test tickets. Features with high values are the words that most consistently influence routing decisions across all tickets.

**\*\*Expected output:\*\*** Technical vocabulary like `graphics`, `windows`, `hardware`, `circuit`, `dos`, `bios` should rank highest ,Ã these are strong discriminative signals between the four teams.

**\*\*In Colab:\*\*** Requires `plotly.io.renderers.default = 'colab'` (set in Cell 1). If the plot does not display, try `fig.show(renderer='colab')` explicitly.

**\*\*Alternative for offline use:\*\***

```

```python
import plotly.offline as pyo
pyo.plot(fig, filename='shap_importances.html', auto_open=True)
```

```

### ### 3.4 SHAP Waterfall for One Ticket

```

```python
fig = explainer.plot_shap_contributions(idx=0)
fig.show()
```

```

**\*\*What this shows:\*\*** For ticket at index 0, how each feature (word) pushes the prediction up or down for the most likely team. Positive bars = evidence for this team. Negative bars = evidence against.

### ### 3.5 Writing the predict\_fn Closure for LIME

This is the most technically demanding part of the XAI task. LIME works by perturbing raw text (removing words) and observing how predictions change. It therefore needs a function that accepts raw strings and returns class probabilities.

**\*\*The function must reproduce the exact TF-IDF preprocessing pipeline:\*\***

```

```python
def predict_fn(texts):
    """
    LIME-compatible inference function for the HumAIne ticket routing
    model.

    Reproduces the full preprocessing pipeline:
    raw text list ,Ůí TF-IDF vectorization ,Ůí probability matrix

    Args:
        texts: list of raw text strings (LIME passes many perturbed
        versions)

    Returns:
        numpy array of shape (len(texts), 4) ,Ůí class probabilities
        Columns correspond to: Hardware, Software, Design, Electronics
    """
    # Step 1: vectorize using the SAME fitted vectorizer from training
    X_sparse = vectorizer.transform(pd.Series(texts).fillna(''))
    # .fillna('') handles None/empty strings that LIME may pass

    # Step 2: convert to dense DataFrame with named columns
    X_df = pd.DataFrame(X_sparse.toarray(), columns=feature_names)
    # .toarray() converts sparse matrix to dense numpy array
    # feature_names provides the column names the model expects

    # Step 3: return probability matrix (NOT class labels)
    return learner.predict_proba(X_df)
    # Shape: (len(texts), 4) ,Ůí one probability per class per text

```

```

... # Rows sum to 1.0

**Critical implementation details:**
- `vectorizer` is the fitted `TfidfVectorizer` from Cell 2 ,Ã¢ it must be
the same object used during training (same vocabulary, same IDF weights).
- `.fillna('')` is essential ,Ã¢ LIME sometimes passes `None` or empty
strings during perturbation.
- `.toarray()` is essential ,Ã¢ `vectorizer.transform()` returns a sparse
matrix; `predict_proba()` needs a dense array or DataFrame.
- `learner.predict_proba()` NOT `learner.predict()` ,Ã¢ LIME needs
probabilities (continuous), not class labels (discrete).
- The function captures `vectorizer`, `feature_names`, and `learner` from
the outer scope automatically ,Ã¢ no extra arguments needed.

```

```

**Verification:**
```python
test_output = predict_fn([test_texts.iloc[0]])
assert test_output.shape == (1, 4)
assert abs(test_output.sum() - 1.0) < 0.01
```

```

3.6 LIME Local Text Explanation

```

```python
Create explainer with HumAIne team names as class labels
lime_exp = LimeTextExplainer(class_names=TEAM_NAMES)
TEAM_NAMES = ['Hardware', 'Software', 'Design', 'Electronics']

Explain one ticket
exp = lime_exp.explain_instance(
 test_texts.iloc[0], # raw text of the ticket (string, NOT
vectorized)
 predict_fn, # the closure from Section 3.5
 num_features=10, # top 10 most influential words to highlight
 top_labels=2 # show explanation for the 2 most probable
teams
)

```

```

Display with highlighted text in the notebook
exp.show_in_notebook(text=True)
```

```

```

**What `show_in_notebook(text=True)` displays:**
- The raw ticket text with words highlighted green (push toward predicted
team) and red (push away)
- A bar chart of word contributions per class
- `text=True` shows the full highlighted passage ,Ã¢ use this, it is more
informative than `text=False`

```

```

**Alternative display if `show_in_notebook` does not render in Colab:**
```python
exp.as_pyplot_figure()
plt.show()

```

...

**\*\*What to expect:\*\*** The top contributing words should be domain-specific technical vocabulary. For a ticket predicted as `Hardware`, words like `bios`, `drive`, `controller`, `interrupt` should show positive contributions.

### ### 3.7 Connection Between AL and XAI in HumAIne

In HumAIne's design, AL and XAI are complementary:

- AL determines **\*\*which tickets to label\*\*** (uncertainty-driven sampling)
- XAI determines **\*\*why the model made a decision\*\*** (SHAP for global, LIME for local)

The LIME explanation can inform the AL strategy: if the model consistently focuses on the wrong words for certain ticket types, that signals which regions of the input space need more labeled examples, a form of human-guided AL refinement that aligns with HumAIne's HITL paradigm.

---

## ## Part 4 , a HAIC Logging Schema

### ### 4.1 Session Log Structure (schema v1.1)

Each student's session produces a JSON file:

`haic\_log\_<StudentID>\_<Group>.json`

```json

```
{
  "schema_version": "1.1",
  "session": {
    "student_id": "S03",
    "group": "A",
    "cohort": "MSc",
    "session_start": "<ISO timestamp>",
    "session_end": "<ISO timestamp>"
  },
  "events": [
    { "event": "session_start", "timestamp": "..."} ,
    {
      "event": "al_configuration",
      "timestamp": "...",
      "classifier": "LogisticRegression",
      "query_strategy": "entropy_sampling",
      "n_queries_per_round": 10,
      "al_config_time_s": 322.56
    },
    {
      "event": "round_complete",
      "timestamp": "...",
      "round": 1,
      "labeled_so_far": 22,
```

```

        "pool_remaining": 2147,
        "accuracy": 0.6813,
        "config_time_s": 322.56
    },
    { "event": "round_complete", "round": 2, "accuracy": 0.7241,
"delta_accuracy": 0.0428 },
    "... (rounds 3-10)",
    {
        "event": "xai_configuration",
        "timestamp": "...",
        "shap_backend": "LinearExplainer",
        "xai_config_time_s": 248.3,
        "predict_fn_output_shape": [1, 4],
        "predicted_team_ticket0": "Hardware"
    },
    {
        "event": "session_complete",
        "timestamp": "...",
        "final_al_accuracy": 0.8334,
        "xai_complete": true,
        "total_config_time_s": 570.86
    }
],
"summary": {
    "classifier": "LogisticRegression",
    "query_strategy": "entropy_sampling",
    "n_queries_per_round": 10,
    "al_config_time_s": 322.56,
    "n_rounds": 10,
    "total_labels_used": 112,
    "baseline_accuracy": 0.5821,
    "final_al_accuracy": 0.8334,
    "al_improvement": 0.2513,
    "accuracy_history": [0.5821, 0.6813, 0.7241, 0.7589, 0.7812, 0.7994,
0.8103, 0.8197, 0.8256, 0.8301, 0.8334],
    "labeled_counts": [12, 22, 32, 42, 52, 62, 72, 82, 92, 102, 112],
    "shap_backend": "LinearExplainer",
    "xai_config_time_s": 248.3,
    "total_config_time_s": 570.86,
    "total_session_time_s": 2738.4
}
}
...

```

4.2 KPI-2.3 Calculation from Logs

```

```python
import json, glob, numpy as np

logs = []
for path in glob.glob('haic_log_*.json'):
 with open(path) as f:
 logs.append(json.load(f))

```

```

group_a = [l['summary']['total_config_time_s'] for l in logs if
l['session']['group'] == 'A']
group_b = [l['summary']['total_config_time_s'] for l in logs if
l['session']['group'] == 'B']

kpi_23 = (np.mean(group_b) - np.mean(group_a)) / np.mean(group_a) * 100
print(f"KPI-2.3 = {kpi_23:.1f}% (target: >=300%)")
print(f"Group A mean: {np.mean(group_a):.0f}s | Group B mean:
{np.mean(group_b):.0f}s")
` ``

```

---

## ## Part 5 ,Ã Notebook Variable Reference

The following variables are defined in Cell 2 of the hackathon notebook and should be assumed available in any code the chatbot generates for this context.

Variable	Type	Shape / Value	Description
`TEAM_NAMES`	list	`['Hardware', 'Software', 'Design', 'Electronics']`	Class names
`CATEGORIES`	list	4 newsgroup category strings	Source data categories
`vectorizer`	`TfidfVectorizer`	fitted, 3000 features	Shared between AL and LIME
`feature_names`	numpy array	shape `(3000,)`	TF-IDF vocabulary terms
`X_train_full`	numpy array	shape `(n_train, 3000)`	All training features
`X_test_np`	numpy array	shape `(n_test, 3000)`	Test features (numpy)
`X_test_df`	pandas DataFrame	shape `(n_test, 3000)`	Test features (named columns, for XAI)
`X_test`	numpy array	alias for `X_test_np`	Used in AL cells
`y_test`	numpy array	shape `(n_test,)`	Test labels (integers 0-3)
`y_test_series`	pandas Series	shape `(n_test,)`	Test labels (for XAI)
`train_texts`	pandas Series	shape `(n_train,)`	Raw training text
`test_texts`	pandas Series	shape `(n_test,)`	Raw test text (for LIME)
`X_initial`	numpy array	shape `(12, 3000)`	Initial 12 labeled samples
`y_initial`	numpy array	shape `(12,)`	Initial 12 labels
`X_pool`	numpy array	shape `(n_pool, 3000)`	Unlabeled pool (shrinks each round)
`y_pool`	numpy array	shape `(n_pool,)`	Pool labels for simulation
`N_INITIAL`	int	`12`	Size of initial labeled seed
`learner`	`ActiveLearner`	,Ã	Created in Cell 3
`MY_CLASSIFIER`	sklearn estimator	,Ã	Chosen in Cell 3

```
| `MY_QUERY_STRATEGY` | callable | ,Ã | Chosen in Cell 3 | |
| `N_QUERIES_PER_ROUND` | int | ,Ã | Chosen in Cell 3 |
| `explainer` | `ClassifierExplainer` | ,Ã | Created in Cell 5 |
| `predict_fn` | callable | ,Ü | shape `(n, 4)` | Created in Cell 5 |
```

---

## Part 6 ,Ã Frequently Asked Questions (Anticipated Student Queries)

### Q: Which classifier should I use for text classification with Active Learning?

\*\*A:\*\* Use `LogisticRegression(max\_iter=1000)`. TF-IDF features are high-dimensional and sparse ,Ã LogisticRegression handles this well, is fast, and supports `predict\_proba()` which is required for uncertainty-based query strategies. In HumAIne's architecture, it also enables the fast `LinearExplainer` SHAP backend in the XAI step.

### Q: Which query strategy should I use for 4 classes with only 12 initial labels?

\*\*A:\*\* Use `entropy\_sampling`. With multiple classes and very few initial labels, you need a strategy that captures uncertainty across all class probabilities simultaneously. Entropy sampling computes Shannon entropy over the full probability distribution ,Ã it is the most complete measure of uncertainty for multi-class problems.

### Q: How do I initialize an ActiveLearner with LogisticRegression and entropy\_sampling?

```
A:
```python
from modAL.models import ActiveLearner
from modAL.uncertainty import entropy_sampling
from sklearn.linear_model import LogisticRegression

learner = ActiveLearner(
    estimator=LogisticRegression(max_iter=1000),
    query_strategy=entropy_sampling,
    X_training=X_initial,
    y_training=y_initial
)
...

```

Q: How do I write one query-teach cycle with modAL?

```
**A:**
```python
Query
query_idx, query_instance = learner.query(X_pool,
n_instances=N_QUERIES_PER_ROUND)
Retrieve labels
new_labels = y_pool[query_idx]
Teach
learner.teach(X_pool[query_idx], new_labels)
Remove from pool
X_pool = np.delete(X_pool, query_idx, axis=0)
y_pool = np.delete(y_pool, query_idx, axis=0)

```

```
...
```

```
Q: How do I wrap my trained learner in a ClassifierExplainer?
A: Pass `learner.learner`, not `learner`, Æ this ensures the fast
`LinearExplainer` SHAP backend is selected:
```

```
```python  
from explainerdashboard import ClassifierExplainer
```

```
explainer = ClassifierExplainer(  
    learner.learner,  
    X_test_df,  
    y_test_series,  
    labels=TEAM_NAMES  
)  
...
```

```
### Q: How do I write a predict_fn for LIME on TF-IDF text data?
```

```
**A:**  
```python  
def predict_fn(texts):
 X_sparse = vectorizer.transform(pd.Series(texts).fillna(''))
 X_df = pd.DataFrame(X_sparse.toarray(), columns=feature_names)
 return learner.predict_proba(X_df)
...
```

The function must use `fillna('')` (handles None), `.toarray()` (converts sparse to dense), and `predict\_proba` (not `predict`).

```
Q: How do I show the SHAP feature importance plot in Colab?
```

```
A:
```python  
fig = explainer.plot_importances()  
fig.show()  
...
```

The plotly renderer is set to 'colab' in Cell 1. If the plot does not render, try `fig.show(renderer='colab')` explicitly.

```
### Q: How do I run LIME and display the explanation?
```

```
**A:**  
```python  
from lime.lime_text import LimeTextExplainer

lime_exp = LimeTextExplainer(class_names=TEAM_NAMES)
exp = lime_exp.explain_instance(
 test_texts.iloc[0],
 predict_fn,
 num_features=10,
 top_labels=2
)
exp.show_in_notebook(text=True)
...
```

```
Q: What is the connection between Active Learning and XAI in HumAIne?
```

```
A: In HumAIne's HITL architecture, AL and XAI serve complementary
roles. AL determines which samples to label next (based on model
```

uncertainty), while XAI explains why the model made a particular decision. The LIME local explanations can directly inform the AL strategy: if the model focuses on irrelevant words for certain ticket types, that region of the input space needs more labeled examples ,Ã a form of human-guided active learning that embodies the Human-in-the-Loop principle.

---

\*HumAIne EU Project ,Ã Grant Agreement 101120218\*

\*Document version: 1.1 | Covers: AL + XAI combined session\*

\*For RAG indexing into HumAIne Swarm chatbot\*